06 · Problem 1: Two Models to Run and Maintain

Step 1: What happens

Every search needs **two** large neural networks: the LLM (to write the query) and the embedding model (to encode it). In the paper's setup both are 8-billion-parameter models: Qwen3-8B and Qwen3-Embedding-8B.

Step 2: Why it hurts

Cost	Detail
Hardware	Both models need GPU memory. An 8B model in 16-bit precision needs roughly 16 GB just for its weights.
Serving	Two services to deploy, monitor, scale and secure. If either one fails, search fails.
Versioning	The index is tied to one exact embedder. Upgrading the embedder means re-embedding every document.
Money	Two models to host, or two API bills.

Step 3: A concrete example

A company runs an internal assistant. The LLM sits on one GPU server and the embedder on another. One week the embedding service is slow because another team is bulk-indexing documents on it. The assistant's answers slow down too, even though the LLM itself is fine. Nothing about the conversation changed; a second dependency got in the way.

Step 4: How people patched it

- **Smaller embedders** save money but tend to search worse.
- **Hosted embedding APIs** move the work elsewhere; the dependency (and the network round trip) remains.
- **Using the same model family** for both jobs (as the paper does) simplifies tooling, but it is still two separate models.

Step 5: What a real fix would need

A way to get an index-compatible query vector **from the LLM alone**, without a second large network at search time, while keeping the existing index usable (no re-embedding of documents).

→ Answered by solution steps `11` (reuse the LLM) and `14` (copy the teacher's space).